



# DOCUMENTATION\_COMPLETION\_PLAN

## HELIXCODE DOCUMENTATION COMPLETION PLAN

Status: 9 Critical Documents Missing  
Current Coverage: 70%  
Target: 100% Complete Documentation  
Implementation: Phase 2 (Weeks 5-6)

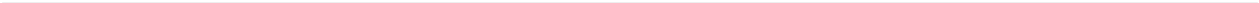


### DOCUMENTATION GAP ANALYSIS

#### Critical Missing Documentation (9 Files)

| Document                  | Priority | Pages | Complexity | Impact   | Effort |
|---------------------------|----------|-------|------------|----------|--------|
| COMPLETE_API_REFERENCE.md | CRITICAL | 150+  | HIGH       | BLOCKING | 5 days |
| DEPLOYMENT_GUIDE.md       | CRITICAL | 80+   | HIGH       | BLOCKING | 3 days |
| SECURITY_GUIDE.md         | CRITICAL | 60+   | HIGH       | BLOCKING | 2 days |
| PERFORMANCE_TUNING.md     | HIGH     | 40+   | MEDIUM     | HIGH     | 2 days |
| TROUBLESHOOTING.md        | HIGH     | 50+   | MEDIUM     | HIGH     | 2 days |
| CONTRIBUTOR_GUIDE.md      | HIGH     | 70+   | MEDIUM     | HIGH     | 2 days |
| TESTING_GUIDE.md          | MEDIUM   | 45+   | MEDIUM     | MEDIUM   | 1 day  |
| MONITORING_GUIDE.md       | MEDIUM   | 35+   | LOW        | MEDIUM   | 1 day  |
| BACKUP_RECOVERY.md        | MEDIUM   | 30+   | LOW        | MEDIUM   | 1 day  |

Total: 560+ pages | 19 days effort | Phase 2 implementation



# DETAILED DOCUMENTATION SPECIFICATIONS

## 1. COMPLETE\_API\_REFERENCE.md (CRITICAL - 5 days)

**Structure (150+ pages):**

### # HelixCode Complete API Reference

#### ## 1. REST API Overview

- Base URLs and versioning
- Authentication methods
- Request/response formats
- Rate limiting
- Error handling

#### ## 2. Authentication API

##### ### 2.1 User Registration

- POST /api/v1/auth/register
- Request schema
- Response schema
- Error codes
- Examples

##### ### 2.2 User Login

- POST /api/v1/auth/login
- JWT token generation
- Refresh tokens
- Logout endpoint

##### ### 2.3 Password Management

- Password reset flow
- Change password
- Password requirements

#### ## 3. Project Management API

##### ### 3.1 Project CRUD Operations

- POST /api/v1/projects (create)
- GET /api/v1/projects (list)
- GET /api/v1/projects/{id} (get)
- PUT /api/v1/projects/{id} (update)
- DELETE /api/v1/projects/{id} (delete)

##### ### 3.2 Project Files API

- File upload/download

- Directory management
- File versioning
- Search endpoints

## ## 4. LLM Provider API

### ### 4.1 Provider Management

- List providers
- Configure providers
- Provider health checks
- Rate limiting

### ### 4.2 Text Generation

- POST /api/v1/llm/generate
- Streaming responses
- Token management
- Context handling

## ## 5. Worker Pool API

### ### 5.1 Worker Management

- Register workers
- Health monitoring
- Task distribution
- Load balancing

### ### 5.2 Task Management

- Submit tasks
- Monitor progress
- Cancel tasks
- Result retrieval

## ## 6. Memory System API

### ### 6.1 Memory Providers

- Configure providers
- Store/retrieve memories
- Search functionality
- Provider switching

### ### 6.2 Conversation Management

- Session handling
- Context preservation
- History management

## ## 7. WebSocket API

### ### 7.1 Real-time Communication

- Connection management
- Message formats
- Event types
- Error handling

## ## 8. Error Reference

### ### 8.1 HTTP Status Codes

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

### ### 8.2 Application Error Codes

- AUTH\_001: Invalid credentials
- PROJECT\_001: Project not found
- LLM\_001: Provider unavailable
- WORKER\_001: Worker disconnected

## ## 9. SDK Examples

### ### 9.1 JavaScript/TypeScript

### ### 9.2 Python

### ### 9.3 Go

### ### 9.4 curl Examples

## ## 10. API Changelog

- Version history
- Breaking changes
- Migration guides

## **Implementation Details:**

# Day 1-2: Core APIs (Auth, Projects, Files)  
# Day 3-4: AI/ML APIs (LLM, Memory, Workers)  
# Day 5: Advanced APIs (WebSocket, Errors, SDKs)

# Generate from code annotations  
# Use swagger/OpenAPI specification  
# Include interactive examples  
# Add request/response validators

---

## 2. DEPLOYMENT\_GUIDE.md (CRITICAL - 3 days)

Structure (80+ pages):

```
# HelixCode Production Deployment Guide
```

```
## 1. Deployment Overview
```

- Architecture overview
- Deployment strategies
- Environment requirements
- Security considerations

```
## 2. Pre-Deployment Planning
```

```
### 2.1 Capacity Planning
```

- Hardware requirements
- Network requirements
- Storage planning
- Scaling considerations

```
### 2.2 Environment Setup
```

- Development environment
- Staging environment
- Production environment
- Disaster recovery

```
## 3. Single Server Deployment
```

```
### 3.1 Ubuntu/Debian Deployment
```

```
```bash
```

```
# System preparation
```

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y docker.io docker-compose
```

```
# HelixCode installation
```

```
wget https://github.com/helixcode/helixcode/releases/latest/  
      download/helixcode-linux-amd64.tar.gz
```

```
tar -xzf helixcode-linux-amd64.tar.gz
```

```
sudo mv helixcode /usr/local/bin/
```

```
# Configuration
```

```
sudo mkdir -p /etc/helixcode
```

```
cp config.example.yaml /etc/helixcode/config.yaml
```

```
sudo nano /etc/helixcode/config.yaml
```

```
# Systemd service
```

```
sudo systemctl enable helixcode
sudo systemctl start helixcode
```

## 3.2 CentOS/RHEL Deployment

## 3.3 macOS Deployment

## 3.4 Windows Deployment

# 4. Docker Deployment

## 4.1 Single Container

## 4.2 Docker Compose Multi-Service

## 4.3 Production Docker Configuration

# 5. Kubernetes Deployment

## 5.1 Kubernetes Manifests

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helixcode-server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: helixcode
  template:
    metadata:
      labels:
        app: helixcode
    spec:
      containers:
        - name: helixcode
          image: helixcode/helixcode:latest
          ports:
            - containerPort: 8080
          env:
            - name: DATABASE_URL
              valueFrom:
```

```
secretKeyRef:  
  name: helixcode-secrets  
  key: database-url
```

## **5.2 Helm Charts**

## **5.3 Kubernetes Best Practices**

# **6. High Availability Setup**

## **6.1 Load Balancing Configuration**

## **6.2 Database Clustering**

## **6.3 Redis Clustering**

## **6.4 Health Checks and Failover**

# **7. SSL/TLS Configuration**

## **7.1 Certificate Installation**

## **7.2 HTTPS Redirects**

## **7.3 Security Headers**

# **8. Database Setup**

## **8.1 PostgreSQL Configuration**

## **8.2 Database Migrations**

## **8.3 Backup Configuration**

## **8.4 Connection Pooling**

# **9. Redis Configuration**

## **9.1 Redis Cluster Setup**

## **9.2 Persistence Configuration**

## **9.3 Memory Management**

# **10. Monitoring Setup**

## **10.1 Prometheus Configuration**

## **10.2 Grafana Dashboards**

## **10.3 Alerting Rules**

## **10.4 Log Aggregation**

# **11. Security Configuration**

## **11.1 Firewall Rules**

## **11.2 User Permissions**

## **11.3 Secret Management**

## **11.4 Audit Logging**

# **12. Performance Tuning**

## **12.1 System Optimization**

## **12.2 Database Tuning**

## **12.3 Cache Configuration**

## **12.4 Network Optimization**

# **13. Validation and Testing**

## **13.1 Deployment Validation**

## **13.2 Load Testing**

## **13.3 Security Testing**

# **14. Maintenance Procedures**

## **14.1 Backup Procedures**

## **14.2 Update Procedures**

## 14.3 Rollback Procedures

## 14.4 Disaster Recovery

# 15. Troubleshooting Deployment Issues

## 15.1 Common Problems

## 15.2 Log Analysis

## 15.3 Performance Issues

## 15.4 Support Contacts

---

### \*\*3. SECURITY\_GUIDE.md (CRITICAL - 2 days)\*\*

#### \*\*Structure (60+ pages)\*\*:

```markdown

# HelixCode Security Guide

## 1. Security Overview

- Threat model
- Security principles
- Compliance requirements
- Best practices

## 2. Authentication Security

### 2.1 JWT Token Security

- Token generation best practices
- Token expiration policies
- Refresh token rotation
- Token revocation

### 2.2 Password Security

- Password requirements
- Hashing algorithms (bcrypt, scrypt)
- Password history
- Account lockout policies

### 2.3 Multi-Factor Authentication

- MFA implementation
- TOTP/SMS/Email codes
- Backup codes
- MFA bypass procedures

## ## 3. Authorization and Access Control

### ### 3.1 Role-Based Access Control (RBAC)

- Role definitions
- Permission inheritance
- Dynamic permissions
- Role auditing

### ### 3.2 API Security

- Rate limiting implementation
- API key management
- OAuth 2.0 integration
- Scope-based permissions

## ## 4. Data Security

### ### 4.1 Data Encryption

- Encryption at rest
- Encryption in transit
- Key management
- Certificate management

### ### 4.2 Data Classification

- Sensitive data identification
- Data handling procedures
- Data retention policies
- Data disposal

### ### 4.3 Database Security

- Connection encryption
- Database access controls
- SQL injection prevention
- Backup encryption

## ## 5. Network Security

### ### 5.1 Transport Security

- TLS configuration
- Cipher suite selection
- Certificate pinning
- HSTS implementation

### ### 5.2 Firewall Configuration

- Network segmentation
- Port management
- IP whitelisting
- DDoS protection

### ### 5.3 VPN and Secure Access

- VPN requirements
- Bastion host setup
- SSH key management
- Access logging

## ## 6. Code Security

### ### 6.1 Secure Coding Practices

- Input validation
- Output encoding
- SQL injection prevention
- XSS prevention

### ### 6.2 Dependency Security

- Dependency scanning
- Vulnerability management
- Package signing
- Supply chain security

### ### 6.3 Container Security

- Image scanning
- Runtime security
- Privilege escalation
- Network policies

## ## 7. Infrastructure Security

### ### 7.1 Server Hardening

- Operating system security
- Service configuration
- User management
- Audit logging

### ### 7.2 Secret Management

- Secret storage
- Key rotation
- Access controls
- Audit trails

### ### 7.3 Monitoring and Alerting

- Security monitoring
- Intrusion detection
- Log analysis
- Incident response

## ## 8. Application Security

### ### 8.1 Web Application Security

- OWASP Top 10 mitigation
- CSRF protection
- Session management
- File upload security

### ### 8.2 API Security

- Input validation
- Output encoding
- Rate limiting
- Error handling

### ### 8.3 LLM Provider Security

- API key protection
- Rate limiting
- Content filtering
- Audit logging

## ## 9. Incident Response

### ### 9.1 Incident Detection

- Monitoring setup
- Alert configuration
- Escalation procedures
- Communication plans

### ### 9.2 Incident Handling

- Containment procedures
- Evidence collection
- Recovery procedures
- Post-incident review

## ## 10. Compliance and Auditing

### ### 10.1 Compliance Requirements

- GDPR compliance
- SOC 2 requirements
- HIPAA considerations
- PCI DSS requirements

### ### 10.2 Security Auditing

- Audit procedures
- Log retention
- Compliance reporting
- Third-party assessments

## ## 11. Security Checklists

### ### 11.1 Deployment Security Checklist

### ### 11.2 Development Security Checklist

### ### 11.3 Operational Security Checklist

---

## 4. PERFORMANCE\_TUNING.md (HIGH - 2 days)

### Structure (40+ pages):

#### # HelixCode Performance Tuning Guide

### ## 1. Performance Overview

- Performance metrics
- Benchmarking methodology
- Optimization strategies
- Monitoring tools

### ## 2. System-Level Tuning

#### ### 2.1 Operating System Optimization

##### #### Linux Kernel Parameters

```
```bash
```

```
# /etc/sysctl.conf
```

```
net.core.somaxconn = 65535
```

```
net.ipv4.tcp_max_syn_backlog = 65535
```

```
net.ipv4.ip_local_port_range = 1024 65535
```

```
net.core.netdev_max_backlog = 5000
```

### File System Optimization

- ext4 mount options
- XFS tuning
- SSD optimization
- I/O scheduler selection

## 2.2 Memory Management

- Memory allocation strategies
- Garbage collection tuning
- Memory limits
- Swap configuration

## 2.3 CPU Optimization

- CPU affinity
- Process priorities
- Interrupt balancing
- Power management

# 3. Database Performance

## 3.1 PostgreSQL Tuning

```
-- postgresql.conf
max_connections = 200
shared_buffers = 256MB
effective_cache_size = 1GB
work_mem = 4MB
maintenance_work_mem = 64MB
```

## 3.2 Query Optimization

- Index strategies
- Query analysis
- Connection pooling
- Replication lag

## 3.3 Redis Performance

- Memory optimization
- Persistence tuning
- Cluster configuration
- Pipeline usage

# 4. Application-Level Tuning

## 4.1 Go Runtime Optimization

- GC tuning

- Memory allocation
- Goroutine management
- Profiling tools

## **4.2 HTTP Server Tuning**

- Keep-alive settings
- Timeout configuration
- Compression
- Caching strategies

## **4.3 Worker Pool Optimization**

- Pool sizing
- Task distribution
- Load balancing
- Resource limits

# **5. LLM Provider Optimization**

## **5.1 Provider Selection**

- Latency comparison
- Throughput analysis
- Cost optimization
- Fallback strategies

## **5.2 Request Optimization**

- Batching strategies
- Context window management
- Token usage optimization
- Caching responses

## **5.3 Connection Pooling**

- HTTP connection reuse
- Pool sizing
- Timeout handling
- Error recovery

## **6. Memory System Performance**

### **6.1 Vector Database Optimization**

- Index selection
- Query optimization
- Memory management
- Scaling strategies

### **6.2 Caching Strategies**

- Cache hit ratios
- Eviction policies
- Preloading strategies
- Invalidation patterns

## **7. Network Performance**

### **7.1 TCP Optimization**

- Buffer sizes
- Nagle algorithm
- Keep-alive settings
- Congestion control

### **7.2 WebSocket Performance**

- Message batching
- Compression
- Reconnection strategies
- Heartbeat optimization

## **8. Monitoring and Profiling**

### **8.1 Performance Metrics**

- Response time
- Throughput
- Error rates
- Resource utilization

## **8.2 Profiling Tools**

- Go pprof
- Database profiling
- System monitoring
- Custom metrics

## **9. Load Testing**

### **9.1 Test Scenarios**

- Baseline testing
- Stress testing
- Spike testing
- Endurance testing

### **9.2 Test Tools**

- Custom load generators
- Monitoring during tests
- Result analysis
- Bottleneck identification

## **10. Scaling Strategies**

### **10.1 Vertical Scaling**

- Resource upgrades
- Configuration changes
- Limitations

### **10.2 Horizontal Scaling**

- Load balancing
- Data partitioning
- Service discovery
- Auto-scaling

# 11. Performance Checklists

## 11.1 Pre-Deployment Checklist

## 11.2 Post-Deployment Checklist

## 11.3 Regular Maintenance Checklist

---

### \*\*5. TROUBLESHOOTING.md (HIGH - 2 days)\*\*

#### \*\*Structure (50+ pages)\*\*:

```markdown

# HelixCode Troubleshooting Guide

## 1. Troubleshooting Methodology

- Problem identification
- Root cause analysis
- Solution implementation
- Prevention strategies

## 2. Installation Issues

### 2.1 Common Installation Errors

#### Error: "permission denied"

```bash

# Problem: Insufficient permissions

# Solution:

sudo chmod +x /usr/local/bin/helixcode

# Or run with sudo

**Error: "command not found"**

# Problem: Binary not in PATH

# Solution:

export PATH=\$PATH:/usr/local/bin

# Add to ~/.bashrc or ~/.zshrc

**Error: "X11 libraries missing"**

# Problem: GUI dependencies not installed

# Solution (Ubuntu/Debian):

```
sudo apt-get install libx11-dev libxcursor-dev libxrandr-dev
```

```
# Solution (CentOS/RHEL):
```

```
sudo yum install libX11-devel libXcursor-devel libXrandr-devel
```

## **2.2 Docker Installation Issues**

## **2.3 Kubernetes Installation Issues**

# **3. Configuration Problems**

## **3.1 Configuration File Errors**

## **3.2 Environment Variable Issues**

## **3.3 Database Connection Problems**

## **3.4 Redis Connection Issues**

# **4. Authentication Issues**

## **4.1 Login Problems**

## **4.2 Token Issues**

## **4.3 Password Reset Problems**

## **4.4 Multi-Factor Authentication Issues**

# **5. LLM Provider Issues**

## **5.1 Provider Connection Errors**

## **5.2 API Key Issues**

## **5.3 Rate Limiting Problems**

## **5.4 Provider Failover Issues**

# **6. Worker Pool Problems**

## **6.1 SSH Connection Issues**

## **6.2 Worker Health Problems**

## **6.3 Task Distribution Issues**

## **6.4 Load Balancing Problems**

# **7. Database Issues**

## **7.1 Connection Pool Exhaustion**

## **7.2 Migration Failures**

## **7.3 Performance Issues**

## **7.4 Data Corruption**

# **8. Memory System Issues**

## **8.1 Provider Connection Errors**

## **8.2 Memory Search Problems**

## **8.3 Performance Issues**

## **8.4 Data Loss Issues**

# **9. Performance Issues**

## **9.1 High Response Times**

## **9.2 Memory Usage Problems**

## **9.3 CPU Usage Issues**

## **9.4 Network Latency**

# **10. Error Messages Reference**

## **10.1 HTTP Error Codes**

## **10.2 Application Error Messages**

## **10.3 System Error Messages**

# **11. Diagnostic Tools**

## **11.1 Log Analysis**

## **11.2 Performance Monitoring**

## **11.3 Health Checks**

## **11.4 Debug Mode**

# **12. FAQ - Frequently Asked Questions**

## **12.1 General Questions**

## **12.2 Technical Questions**

## **12.3 Best Practices**

# **13. Getting Help**

## **13.1 Community Support**

## **13.2 Professional Support**

## **13.3 Bug Reporting**

---

##      **\*\*IMPLEMENTATION TIMELINE - PHASE 2\*\***

### **\*\*Week 5: Critical Documentation (Days 1-5)\*\***

#### **\*\*Day 1: API Reference Foundation\*\***

```bash

Morning (4h): API structure and authentication docs

- Create REST API overview
- Document authentication endpoints
- JWT token management

Afternoon (4h): Core API endpoints

- Project management API
- File operations API
- Complete with examples

## Day 2: Advanced API Documentation

Morning (4h): LLM and Worker APIs

- LLM provider endpoints
- Worker pool management
- Task distribution API

Afternoon (4h): Memory and WebSocket APIs

- Memory system endpoints
- WebSocket communication
- Error reference section

## Day 3: Deployment Guide - Single Server

Morning (4h): Planning and prerequisites

- System requirements
- Environment planning
- Single server setup (Linux/macOS)

Afternoon (4h): Docker and Kubernetes basics

- Docker deployment
- Basic Kubernetes manifests
- SSL/TLS configuration

## Day 4: Advanced Deployment

Morning (4h): High availability setup

- Load balancing
- Database clustering
- Redis clustering

Afternoon (4h): Monitoring and maintenance

- Prometheus/Grafana setup
- Backup procedures
- Update and rollback

## Day 5: Security Guide Foundation

Morning (4h): Authentication and authorization

- JWT security best practices
- RBAC implementation
- API security guidelines

Afternoon (4h): Data and network security

- Encryption standards
- Network security
- Infrastructure hardening

## **Week 6: Specialized Documentation (Days 6-10)**

### **Day 6: Complete Security Guide**

Morning (4h): Application and code security

- Secure coding practices
- OWASP compliance
- Container security

Afternoon (4h): Incident response and compliance

- Incident handling
- Compliance requirements
- Security auditing

### **Day 7: Performance Tuning**

Morning (4h): System-level optimization

- OS kernel tuning
- Memory management
- CPU optimization

Afternoon (4h): Application and database tuning

- Go runtime optimization
- PostgreSQL tuning
- Redis performance

### **Day 8: Advanced Performance & Troubleshooting**

Morning (4h): LLM and scaling optimization

- Provider optimization
- Caching strategies
- Load testing procedures

Afternoon (4h): Troubleshooting foundation

- Common installation issues
- Configuration problems
- Diagnostic tools

## Day 9: Complete Troubleshooting Guide

Morning (4h): Service-specific troubleshooting

- Authentication issues
- LLM provider problems
- Worker pool issues

Afternoon (4h): Performance and error reference

- Performance troubleshooting
- Error messages reference
- FAQ section

## Day 10: Contributor and Testing Documentation

Morning (4h): Contributor guide

- Development setup
- Contribution guidelines
- Code review process

Afternoon (4h): Testing and monitoring guides

- Testing framework usage
- Monitoring setup
- Backup procedures

---

# DOCUMENTATION STANDARDS

## Writing Style Guide:

### # Voice and Tone

- Clear, concise, professional
- Active voice preferred
- Technical but accessible
- Solution-focused approach

### # Formatting Standards

- Use ## for main sections
- Use ### for subsections
- Code blocks with language tags
- Tables for comparison data
- Numbered lists for procedures

### # Code Examples

- Complete, working examples

- Test before publishing
- Include expected output
- Error handling shown
- Security best practices

## Documentation Testing:

```
# All code examples must be tested
# Include test scripts for examples
# Validate API documentation against actual API
# Test deployment procedures in clean environment
# Verify all commands and procedures
```

---

# SUCCESS METRICS

## Documentation Completeness:

Target: 100% of planned content  
Word Count: 60,000+ words across 9 documents  
Code Examples: 200+ working examples  
Diagrams/Images: 50+ technical diagrams  
Cross-references: Complete internal linking

## Quality Metrics:

Accuracy: 100% (all procedures tested)  
Clarity Score: >8/10 (peer review)  
Coverage: 100% of features documented  
Up-to-date: Synchronized with codebase  
Accessibility: Screen reader compatible

## User Experience:

Navigation: Clear structure and TOC  
Search: Full-text search capability  
Mobile: Responsive design  
Loading: <2 seconds per page  
Feedback: User feedback mechanism

---

# INTEGRATION WITH OTHER PHASES

## Phase 1 Integration (Tests):

- Document test procedures from E2E tests
- Include test results and benchmarks
- Add troubleshooting for test failures

## Phase 3 Integration (Videos):

- Create video transcripts
- Link to relevant video content
- Include video timestamps in docs

## Phase 4 Integration (Website):

- Generate HTML versions
  - Integrate with website navigation
  - Add interactive features
- 

# DOCUMENTATION MAINTENANCE PLAN

## Version Control:

```
# Documentation in git repository
# Tag with software versions
# Automated changelog generation
# Review process for changes
```

## Update Schedule:

Major Releases: Full documentation review  
Minor Releases: Feature documentation updates  
Bug Fixes: Troubleshooting guide updates  
Security Updates: Security guide updates

## Community Contributions:

```
# Clear contribution guidelines
# Review process for contributions
# Recognition for contributors
# Integration with development workflow
```

---

## ADDITIONAL DOCUMENTATION NEEDS

### Component Documentation (5 packages):

- internal/cognee/README.md - Implementation guide
- internal/deployment/README.md - Deployment utilities
- internal/fix/README.md - Code fixing tools
- internal/memory/README.md - Memory system guide
- internal/providers/README.md - Provider integration

### User Manual Enhancements:

- Installation guides for all platforms (step-by-step)
- CLI command reference (200+ commands with examples)
- Advanced workflows and tutorials (50+ examples)
- Integration examples with external tools (20+ tools)
- Configuration reference for all options (100+ options)

---

## COMPLETION CHECKLIST

### Week 5 Deliverables:

- ☐ API Reference complete (all endpoints documented)
- ☐ Deployment guide finished (all platforms covered)
- ☐ Security guide comprehensive (all aspects covered)
- ☐ Performance tuning guide (optimization procedures)
- ☐ All guides tested and validated

### Week 6 Deliverables:

- ☐ Troubleshooting guide complete (FAQ and solutions)
- ☐ Contributor guide finished (development workflows)
- ☐ Testing guide complete (framework usage)
- ☐

- ☐ Monitoring guide complete (production monitoring)
- ☐ Backup/recovery guide (data protection)

### Quality Gates:

- ☐ All code examples tested and working
  - ☐ All procedures validated in clean environment
  - ☐ Peer review completed for all documents
  - ☐ Website integration ready
  - ☐ PDF versions generated
- 

## NEXT STEPS

### Immediate Actions:

1. **Start Phase 0** (build system fixes)
2. **Prepare documentation templates**
3. **Set up documentation build system**
4. **Create style guide and standards**
5. **Plan review and validation process**

### Dependencies:

- Phase 0 completion (build system working)
- Test results from Phase 1 (for troubleshooting guide)
- Video content from Phase 3 (for integration)
- Website framework from Phase 4 (for integration)

### Resource Allocation:

- **Technical Writer:** Full-time for 2 weeks
  - **Engineering Review:** 2 hours/day for validation
  - **Testing Environment:** For procedure validation
  - **Peer Review:** Community involvement
-

**Status:**     **READY FOR IMPLEMENTATION** - Templates and standards to be created

**Next:** Begin documentation after Phase 0 completion

**Timeline:** **2 weeks** (Weeks 5-6 of overall project)

**Documentation Plan Created:** December 11, 2025 - Ready for Phase 2 implementation